BTS SERVICES INFORMATIQUES AUX ORGANISATIONS

Option SLAM — Session 2026

MCCI Business School

RAPPORT DE PROJET

Épreuve E6 — Réalisation Professionnelle N° RP2

Digitalisation du Système de Gestion des Bons Cadeaux

Application VMS — Voucher Management System

BILOUNGA NKPWE'E Epse FOKOU Ingrid Raphaela

N° candidat : 02542605394

Période de réalisation : Octobre 2025 – Mars 2026

Table des matières

Table des matières 2
1. Introduction 4
2. Présentation du contexte et du besoin 5
2.1 Contexte de l'entreprise 5
2.2 Objectifs du projet 5
3. Organisation du travail 6
3.1 Démarche adoptée 6
3.2 Planning de réalisation 6
3.3 Outils et méthodes utilisés 6
4. Conception de la solution 7
4.1 Architecture générale 7
4.2 Modèle de données 7
4.3 Gestion des rôles utilisateurs 8
5. Réalisation et implémentation 8
5.1 Technologies et bibliothèques 8
5.2 Structure du projet 9
5.3 Le workflow de gestion des bons cadeaux 9
a) Création de la demande 10
b) Validation par l'approbateur 11
c) Génération des bons PDF 11
d) Envoi automatique par email 11
e) Rédemption en magasin 12
5.4 Opérations CRUD 13
5.5 Module de reporting 15
6. Difficultés rencontrées 15
6.1 Positionnement des éléments dans le PDF 15
6.2 Intégration de JavaMail 15
6.3 Gestion des droits selon les rôles dans JavaFX 16
7. Résultats obtenus 16
7.1 Fonctionnalités livrées 16
7.2 Tests réalisés 16
7.3 Déploiement 17
8. Bilan 17
8.1 Ce que j'ai appris 17
8.2 Axes d'amélioration 17
8.3 Compétences développées 17

Annexes 18
Annexe A — Comptes de test..... 18
Annexe B — Ressources du projet 18
Annexe C — Diagrammes UML 18

1. Introduction

Ce rapport présente le projet VMS (Voucher Management System), réalisé dans le cadre de ma formation BTS Services Informatiques aux Organisations, option SLAM. Il s'agit de ma deuxième réalisation professionnelle, évaluée lors de l'épreuve E6.

Le projet consiste à développer une application de bureau en JavaFX permettant de gérer intégralement le cycle de vie de bons cadeaux d'entreprise, depuis la création d'une demande jusqu'à l'encaissement en point de vente. J'ai conduit ce projet de manière autonome sur une période de six mois, d'octobre 2025 à mars 2026.

Ce rapport décrit les étapes de ma démarche: analyse du besoin, conception de la solution, choix techniques, mise en œuvre, difficultés rencontrées et résultats obtenus.

2. Présentation du contexte et du besoin

2.1 Contexte de l'entreprise

L'application VMS a été développée pour répondre au besoin d'une organisation exploitant plusieurs points de vente, désignée sous le nom VMS (Vente Multi-Enseignes). Avant ce projet, la gestion des bons cadeaux reposait entièrement sur des supports papier : les commandes étaient saisies à la main, les validations se faisaient de manière informelle, et le suivi des bons distribués était assuré par tableur.

Ce fonctionnement présentait de nombreuses limites. Les bons papier étaient facilement falsifiables et il n'existait aucun mécanisme empêchant qu'un même bon soit présenté plusieurs fois en caisse. Les délais entre la commande et la remise des bons au client pouvaient atteindre plusieurs jours. Enfin, en cas de litige ou d'audit, il était impossible de reconstituer l'historique précis des opérations.

2.2 Objectifs du projet

La mission qui m'a été confiée était d'informatiser intégralement ce processus. Les principaux objectifs définis au départ étaient les suivants :

- Mettre en place un workflow de validation structuré avec des rôles utilisateurs distincts
- Générer automatiquement les bons cadeaux sous forme de PDF sécurisés avec QR code unique
- Envoyer ces bons par email au client dès validation, sans intervention manuelle
- Permettre la rédemption des bons en magasin par simple scan du QR code
- Garantir la traçabilité complète de toutes les opérations via un journal d'audit
- Éliminer le risque de double utilisation d'un même bon

3. Organisation du travail

3.1 Démarche adoptée

Travaillant seule sur ce projet, j'ai adopté une organisation en phases successives. J'ai commencé par une phase d'analyse et de conception avant d'entrer dans le développement proprement dit. Cette approche m'a permis d'avoir une vision claire de ce que je devais produire avant d'écrire la moindre ligne de code.

Les premières semaines ont été consacrées à la compréhension du besoin, à la modélisation de la base de données et à la conception des diagrammes UML. J'ai ensuite développé les fonctionnalités en commençant par les fondations de l'application : la connexion à la base de données, le système d'authentification et la gestion des rôles. Les fonctionnalités métier ont été développées progressivement, dans l'ordre logique du workflow: création des demandes, validation, génération des PDF, envoi email, puis rédemption en magasin.

3.2 Planning de réalisation

Le tableau ci-dessous résume les grandes phases du projet sur les six mois de réalisation :

Période	Travaux réalisés
Octobre 2025	Analyse du besoin, conception de la base de données, diagrammes UML
Novembre 2025	Mise en place du projet Maven, architecture MVC/DAO, authentification
Décembre 2025	Gestion des utilisateurs et des rôles, interfaces JavaFX principales
Janvier 2026	Workflow de demande et validation, notifications email
Février 2026	Génération PDF avec QR codes, module de rédemption en magasin
Mars 2026	Module de reporting, exports Excel/CSV, tests, corrections, documentation

3.3 Outils et méthodes utilisés

Pour gérer mon code source, j'ai utilisé Git avec un dépôt hébergé sur GitHub. Cela m'a permis de versionner mon travail et de revenir sur des versions antérieures en cas de problème. Le dépôt contient également la documentation du projet et l'exécutable de l'application.

Pour la conception des interfaces graphiques, j'ai utilisé Scene Builder, qui permet de créer les fichiers FXML de manière visuelle. IntelliJ IDEA a été mon environnement de développement principal tout au long du projet.

4. Conception de la solution

4.1 Architecture générale

La solution s'articule autour de trois composantes principales qui communiquent toutes avec une base de données PostgreSQL centralisée. L'application siège est utilisée au bureau pour créer et gérer les demandes de bons cadeaux. L'application magasin est utilisée aux points de vente pour effectuer les rédemptions. Ces deux applications partagent la même base de données, ce qui garantit que les informations sont cohérentes et à jour en temps réel sur tous les postes.

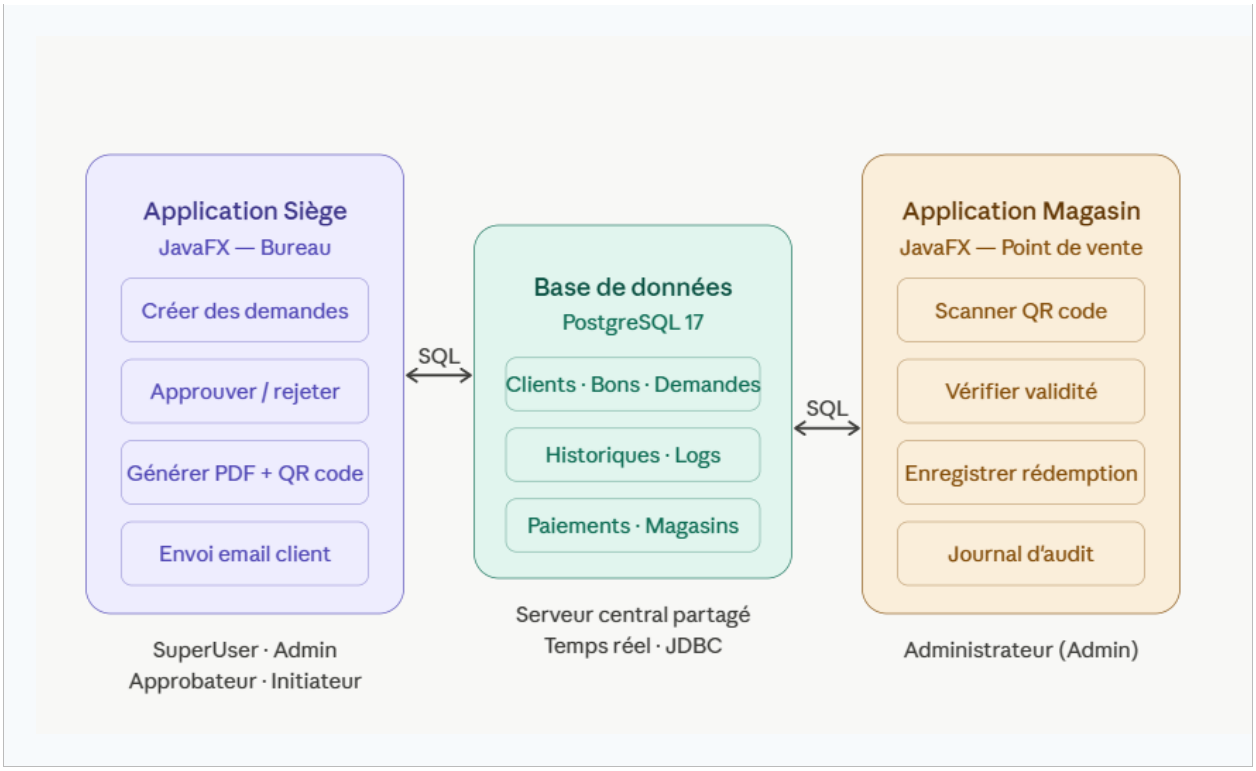

Figure 1 — Architecture de l'application VMS

L'architecture logicielle adoptée est le pattern MVC (Modèle-Vue-Contrôleur) associé au pattern DAO (Data Access Object). Cette séparation des responsabilités m'a permis de structurer le code de manière claire et de faciliter la maintenance. Les classes du modèle représentent les entités métier, les fichiers FXML définissent les vues, les contrôleurs gèrent les interactions, et les classes DAO assurent exclusivement les accès à la base de données.

4.2 Modèle de données

La base de données PostgreSQL a été conçue pour stocker l'ensemble des informations du système. Les tables principales sont les suivantes : la table des utilisateurs avec leurs rôles et mots de passe sécurisés, la table des demandes représentant les commandes de bons cadeaux, la table des vouchers correspondant aux bons individuels générés, la table des clients, la table des magasins, la table des rédemptions enregistrant chaque utilisation d'un bon, et enfin une table de logs d'audit.

J'ai également créé un trigger PostgreSQL sur la table des rédemptions. Ce trigger se déclenche automatiquement si quelqu'un tente de scanner un bon déjà marqué comme utilisé. Il enregistre un log d'alerte dans la table d'audit, ce qui permet de détecter d'éventuelles tentatives de fraude.

4.3 Gestion des rôles utilisateurs

L'application distingue quatre profils d'utilisateurs, chacun disposant d'un périmètre d'action défini. Le SuperUser dispose de tous les droits et peut notamment administrer les comptes utilisateurs. L'Initiateur est le collaborateur chargé de saisir les demandes de bons cadeaux suite aux commandes clients. L'Approbateur reçoit une notification par email et doit valider ou rejeter chaque demande. Enfin, le rôle Admin correspond aux superviseurs en magasin qui effectuent les rédemptions.

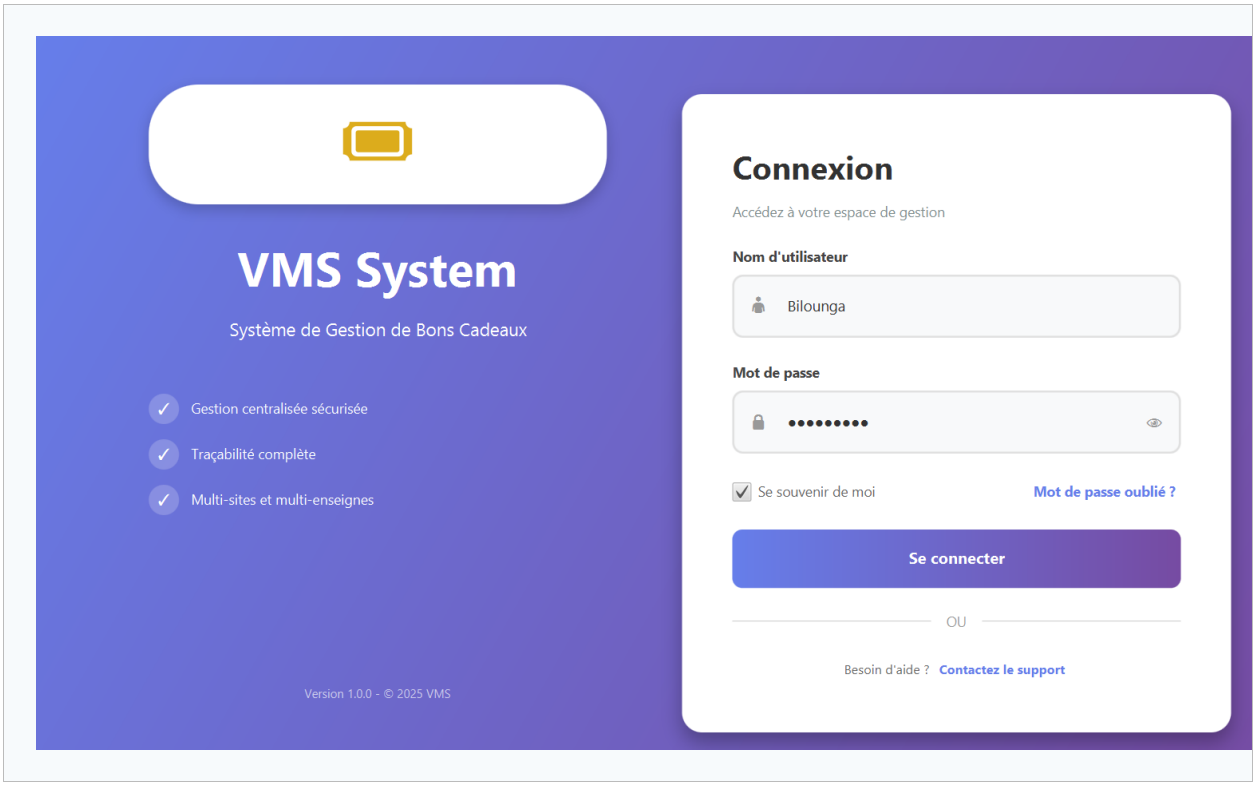

Figure 3 — Interface de connexion avec sélection du rôle

5. Réalisation et implémentation

5.1 Technologies et bibliothèques

L'application a été développée en Java 17 avec le framework JavaFX 17 pour les interfaces graphiques. La base de données PostgreSQL est accessible depuis l'application via JDBC. Maven est utilisé comme gestionnaire de dépendances, ce qui facilite l'intégration des bibliothèques tierces.

Plusieurs bibliothèques externes ont été intégrées pour couvrir les besoins spécifiques du projet. Apache PDFBox permet de générer les bons cadeaux en PDF. La bibliothèque ZXing de Google prend en charge la génération des QR codes intégrés dans ces PDF. BCrypt assure le hachage sécurisé des mots de passe avant leur stockage en base de données. Enfin, Apache POI est utilisée pour les exports au format Excel dans le module de reporting.

5.2 Structure du projet

Le projet est organisé en packages distincts conformément à l'architecture MVC/DAO. Le package model contient les classes représentant les entités métier (Voucher, Demande, Utilisateur, Magasin, etc.). Le package dao regroupe les classes d'accès aux données, avec une classe DAO par entité. Le package controller contient les contrôleurs JavaFX associés à chaque écran de l'application. Le package service héberge les services transverses comme l'envoi d'email, la génération de PDF et la création de QR codes. Le package util contient les classes utilitaires, notamment la gestion de la connexion à la base de données.

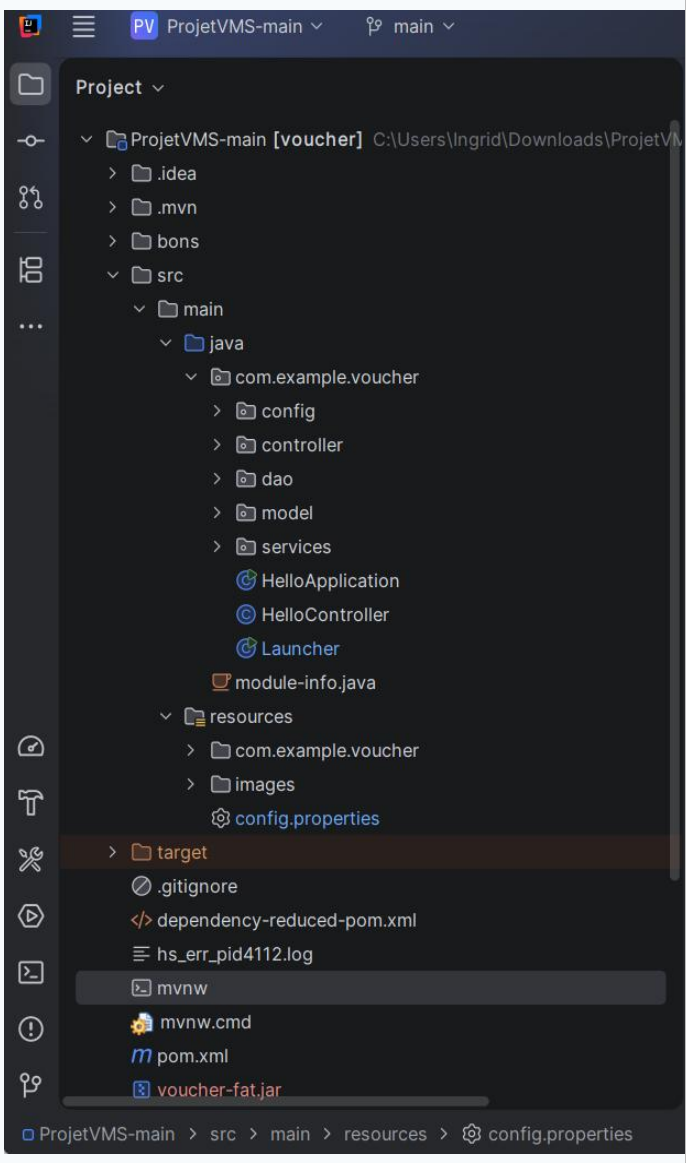

Figure 4 — Organisation des packages du projet

5.3 Le workflow de gestion des bons cadeaux

Le cœur fonctionnel de l'application est le workflow en cinq étapes qui couvre le cycle de vie complet d'un bon cadeau.

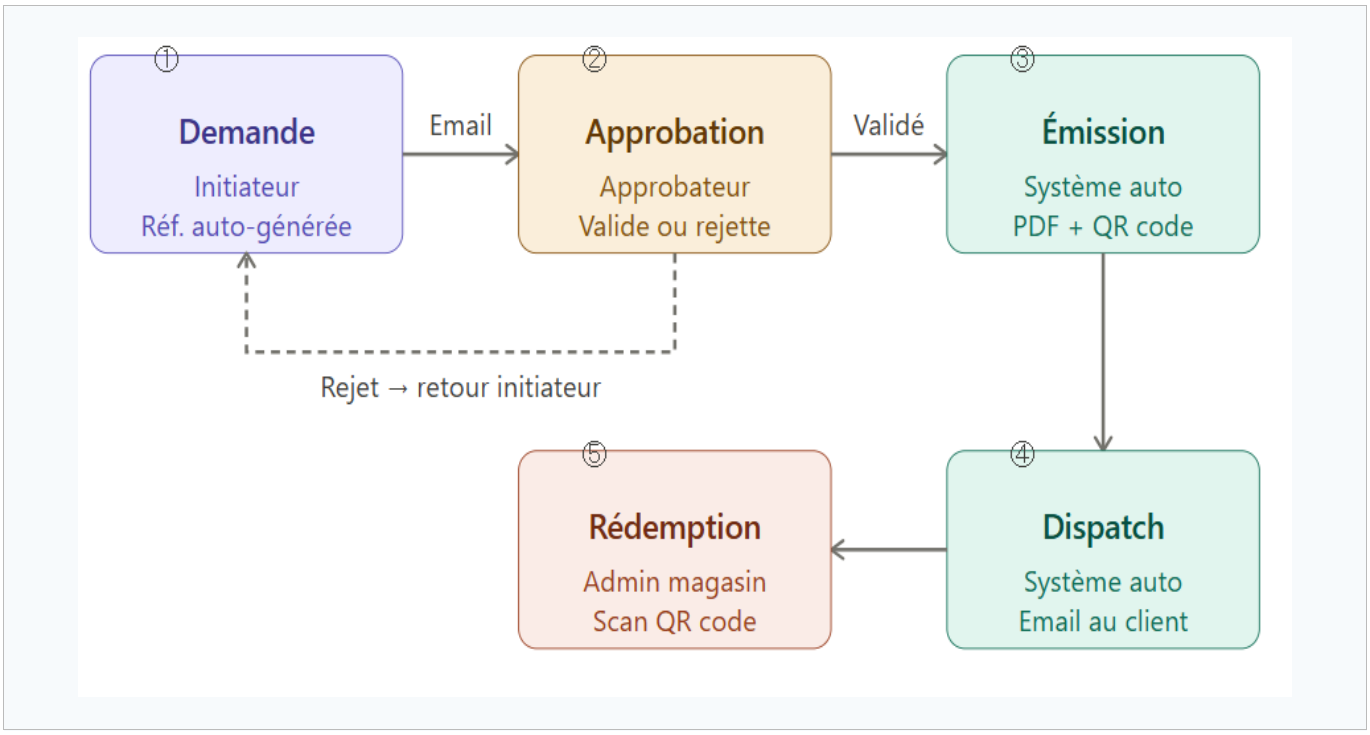

Figure 5 — Workflow complet de gestion d'un bon cadeau

a) Création de la demande

L'initiateur saisit une nouvelle demande dans le formulaire dédié. Il renseigne le nom du client, la valeur souhaitée pour chaque bon, le nombre de bons et la durée de validité. À la validation, le système génère automatiquement une référence unique pour la demande et envoie un email de notification aux approbateurs.

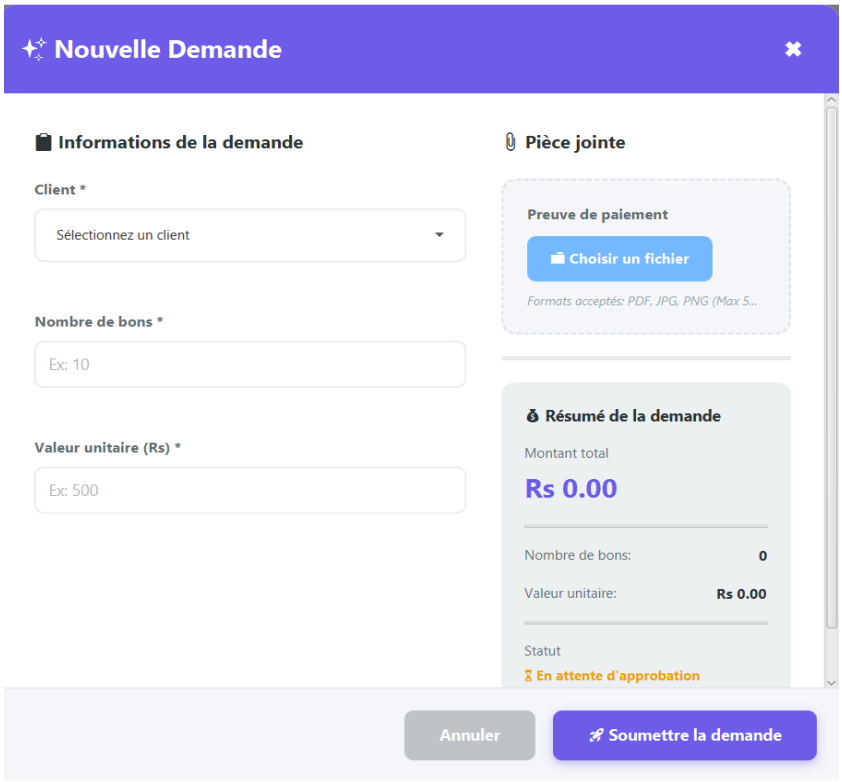

Figure 6 — Formulaire de saisie d'une nouvelle demande

b) Validation par l'approbateur

L'approbateur reçoit un email l'informant qu'une demande est en attente. Il accède à la liste des demandes dans son interface, consulte les détails et peut approuver ou rejeter la demande. En cas de rejet, un motif peut être saisi.

c) Génération des bons PDF

Dès qu'une demande est approuvée, le système génère automatiquement les bons cadeaux. Chaque bon est une page du PDF : si le client a commandé dix bons, le fichier PDF contiendra dix pages. Chaque page affiche le nom du client, la valeur du bon en grand format, un QR code unique, la date d'expiration et une référence textuelle de type VMS-2025-0018-003. Une image de fond personnalisée donne au document une identité visuelle professionnelle.

Figure 8 — Exemple de bon cadeau généré automatiquement

d) Envoi automatique par email

Le PDF est envoyé automatiquement par email au client dès sa génération, sans aucune intervention manuelle. Le service d'envoi d'email utilise JavaMail et le fichier PDF est joint au message. Une confirmation d'envoi est enregistrée dans le système.

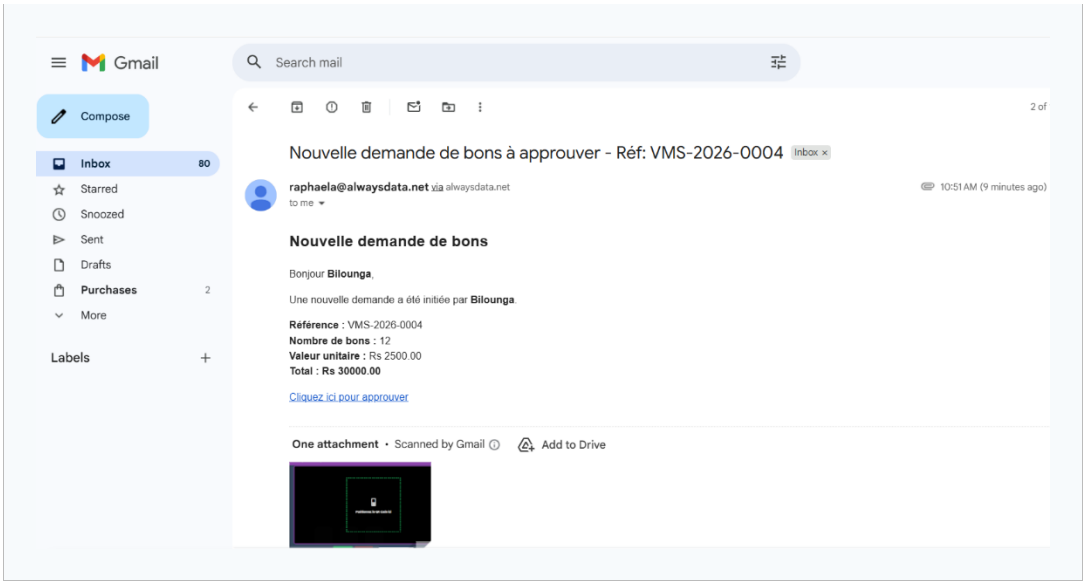

Figure 9 — Confirmation d'envoi automatique du PDF au client

e) Rédemption en magasin

Lorsqu'un client présente son bon en caisse, le superviseur ouvre l'application magasin. Il sélectionne d'abord le magasin concerné, puis clique sur le bouton de rédemption. Une fenêtre s'ouvre permettant soit d'activer la caméra pour scanner le QR code, soit de saisir manuellement le code de référence. L'application interroge immédiatement la base de données pour vérifier que le bon est valide, non expiré et non encore utilisé.

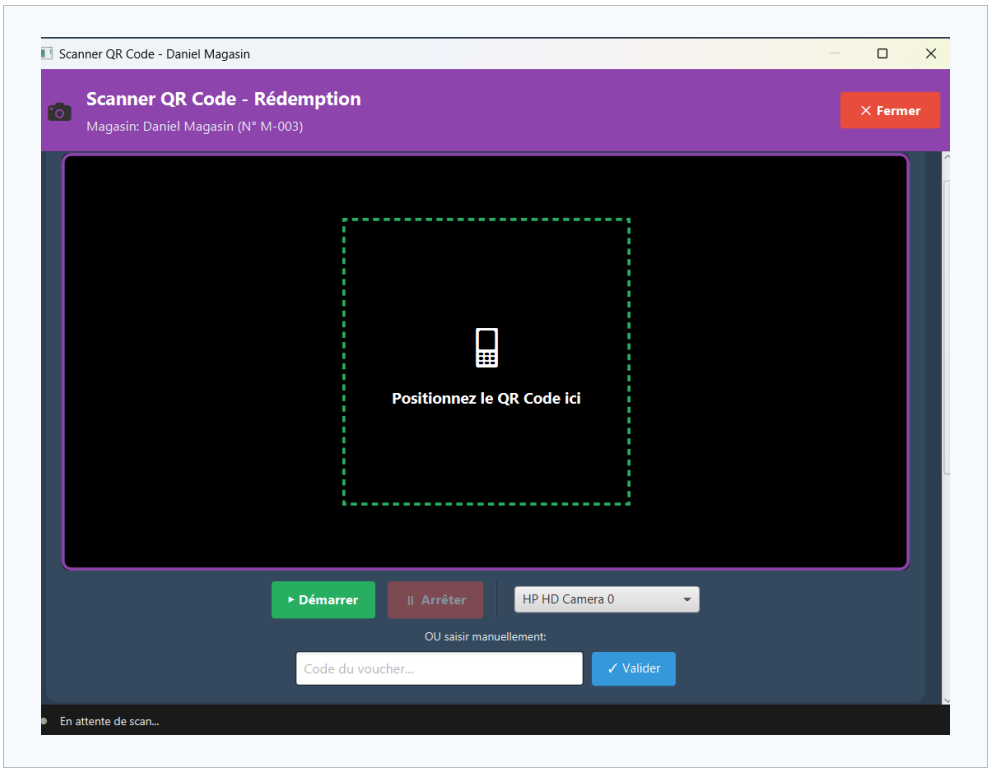

Figure 10 — Module de scan QR code en magasin

Si le bon est valide, un message de confirmation s'affiche et la rédemption est enregistrée. Le statut du bon est immédiatement mis à jour en base de données et toute nouvelle tentative de scan de ce même bon sera rejetée. Si le bon a déjà été utilisé, le système affiche un message d'erreur explicite et le trigger PostgreSQL crée un log d'alerte dans la table d'audit.

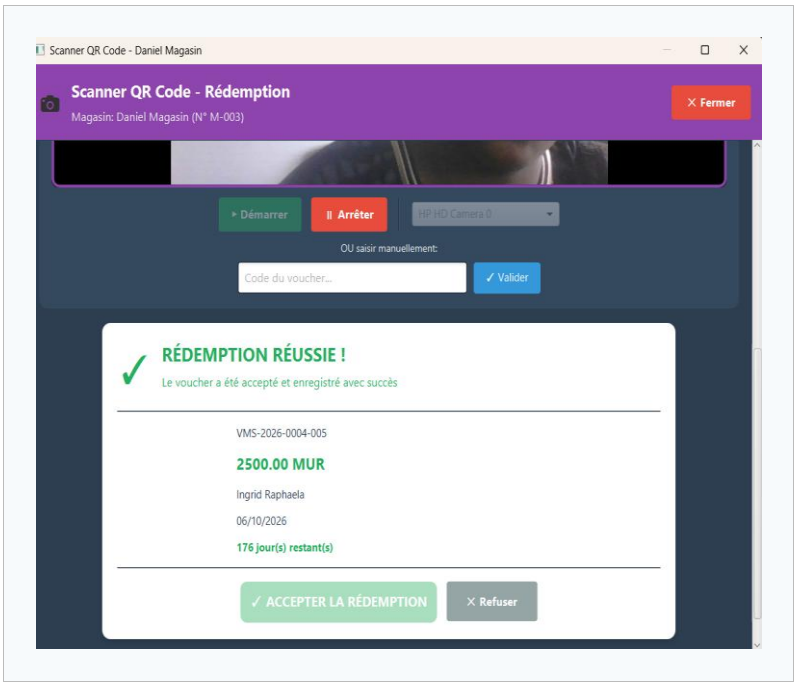

Figure 11 — Confirmation de rédemption valide

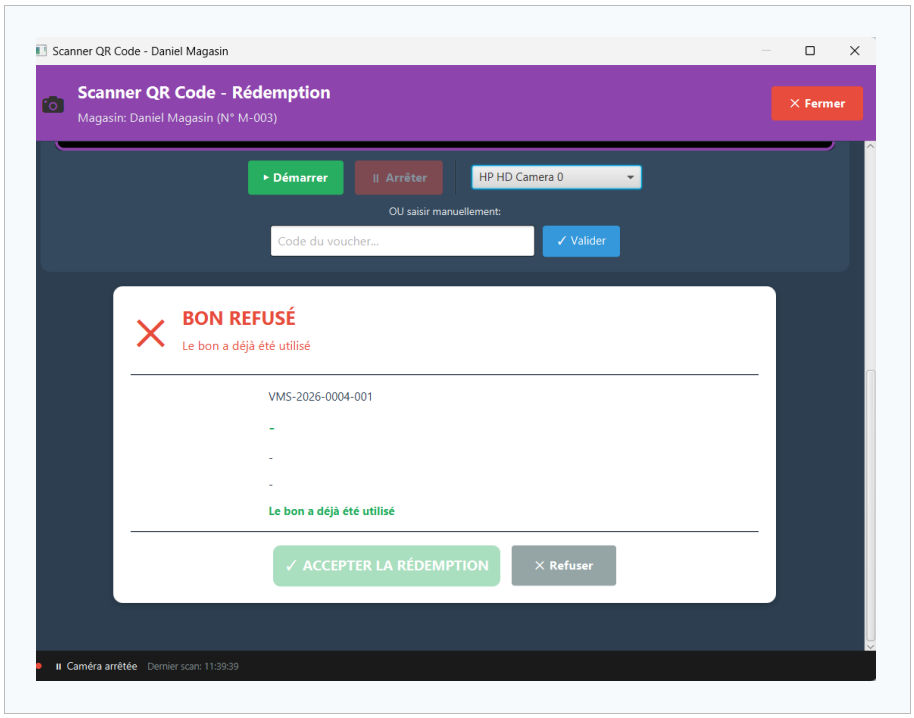

Figure 12 — Rejet d'un bon déjà consommé

5.4 Opérations CRUD

L'application propose des interfaces de gestion complètes pour les principales entités du système. Pour les utilisateurs, le SuperUser peut créer des comptes, modifier les informations, changer le rôle d'un utilisateur, activer ou désactiver un compte et le supprimer. La même logique s'applique aux magasins : il est possible d'en ajouter, de modifier les informations, de les rendre actifs ou inactifs, ou de les supprimer.

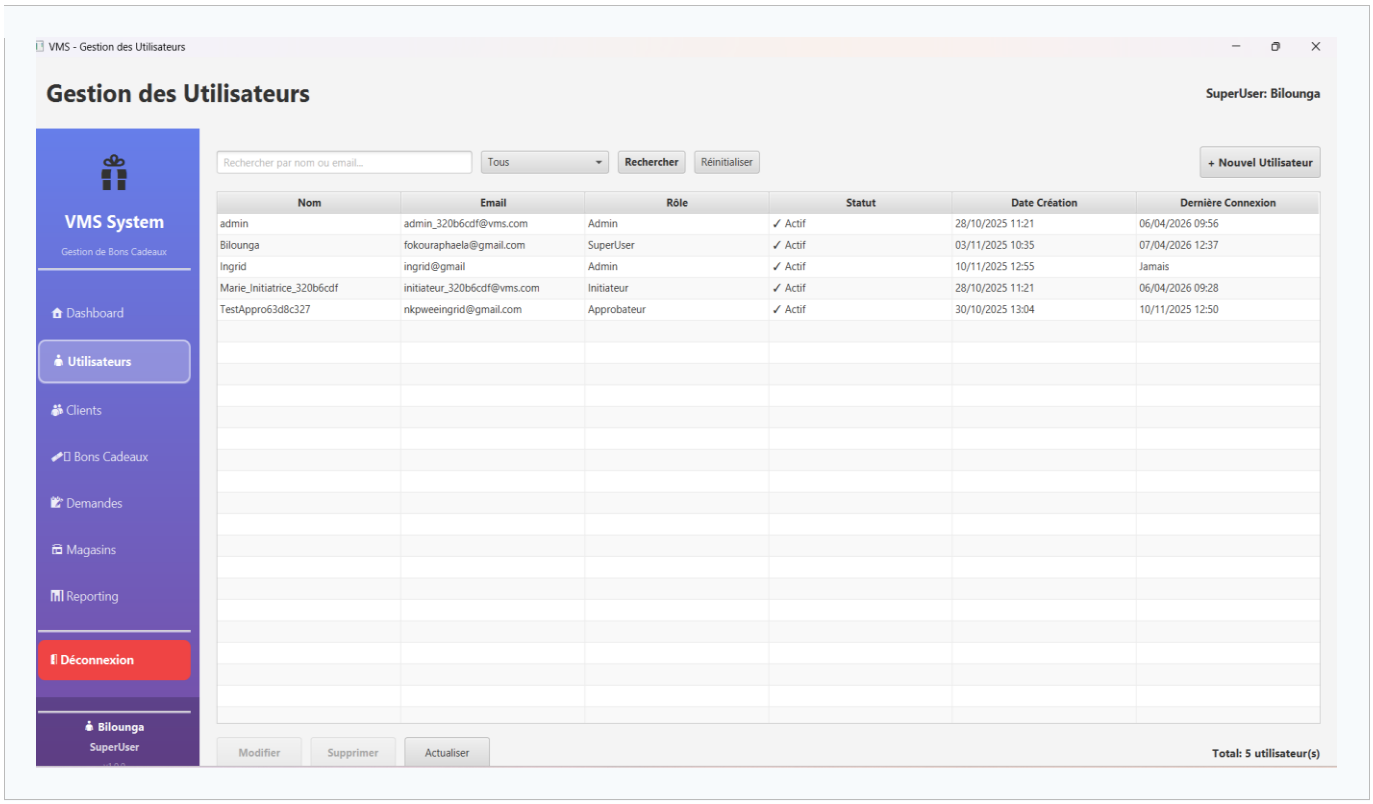

Figure 13 — Interface de gestion des utilisateurs

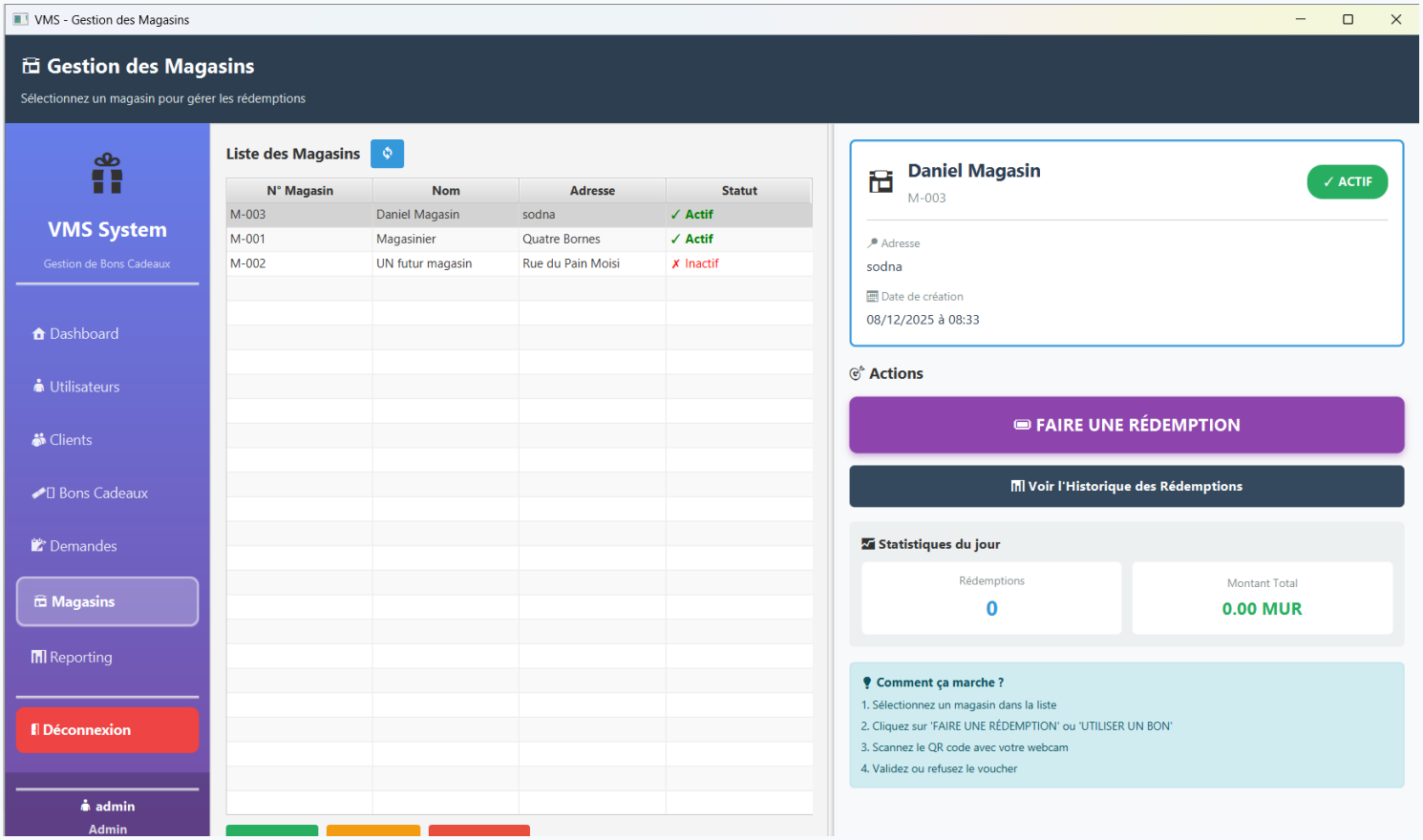

Figure 14 — Interface de gestion des magasins

5.5 Module de reporting

L'application intègre un module de reporting permettant de consulter et d'exporter différents rapports : liste des demandes par statut, historique des rédemptions par magasin, bons arrivant à expiration et journal complet des logs d'audit. Ces rapports peuvent être exportés aux formats Excel et CSV grâce à la bibliothèque Apache POI, ce qui facilite leur exploitation dans d'autres outils.

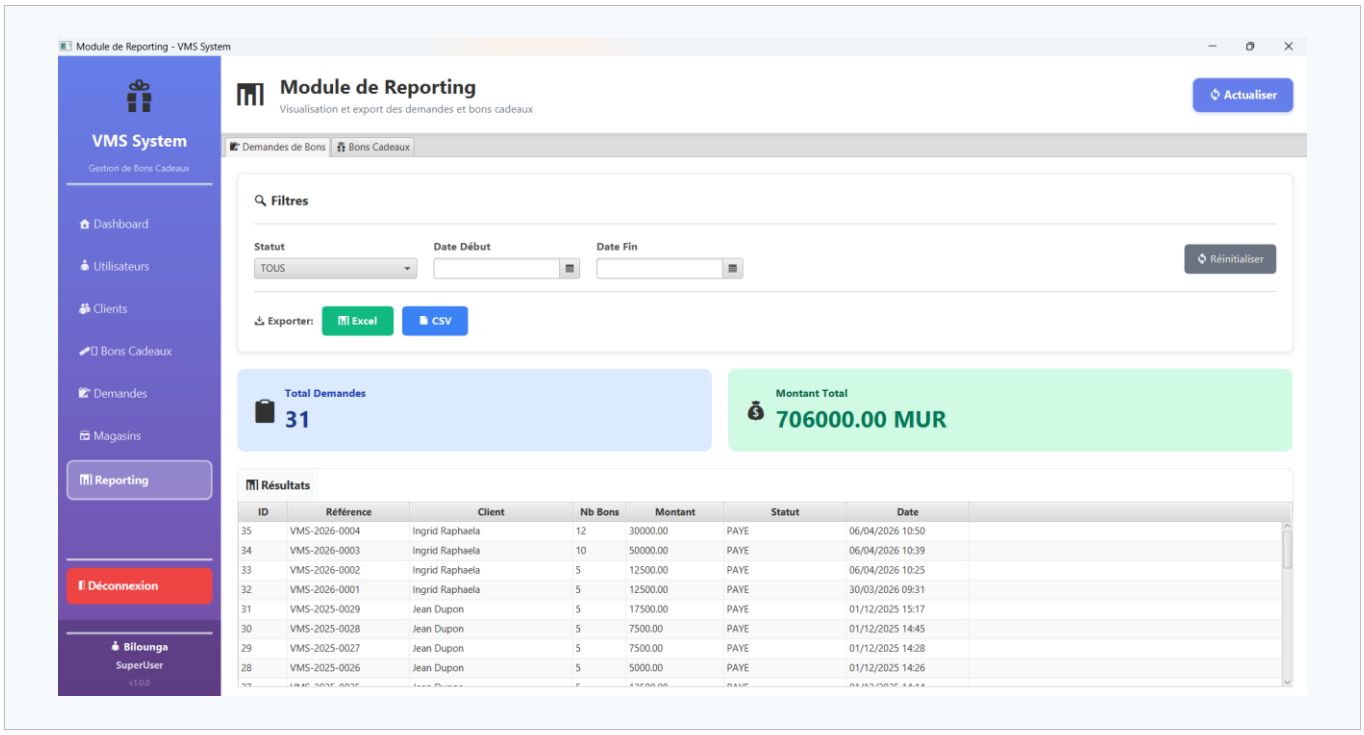

Figure 15 — Module de reporting et export Excel/CSV

6. Difficultés rencontrées

6.1 Positionnement des éléments dans le PDF

L'une des premières difficultés techniques que j'ai rencontrées concernait la génération du PDF avec Apache PDFBox. Positionner précisément le QR code, le texte et l'image de fond sur la page a demandé du temps. Le système de coordonnées de PDFBox est exprimé en points typographiques avec l'origine en bas à gauche de la page, ce qui est différent de ce à quoi on est habitué dans la plupart des interfaces graphiques. J'ai dû faire plusieurs essais pour obtenir un rendu propre et lisible.

6.2 Intégration de JavaMail

La mise en place de l'envoi automatique d'emails a également posé quelques difficultés, notamment la configuration du serveur SMTP et la gestion des pièces jointes volumineuses. Dans les premiers tests, des délais d'attente trop courts causaient des erreurs lors de l'envoi de fichiers PDF contenant de nombreuses pages. J'ai résolu le problème en ajustant les paramètres de délai et en testant avec différents fournisseurs de messagerie.

6.3 Gestion des droits selon les rôles dans JavaFX

Faire en sorte que l'interface affiche uniquement les fonctionnalités accessibles à l'utilisateur connecté a nécessité une réflexion sur la gestion de l'état global de la session. J'ai mis en place une classe de session qui stocke le rôle et les informations de l'utilisateur connecté, et chaque contrôleur consulte cette session au chargement pour masquer ou désactiver les composants non autorisés.

7. Résultats obtenus

7.1 Fonctionnalités livrées

À l'issue du projet, l'application est pleinement fonctionnelle et couvre l'ensemble du périmètre défini au départ. Le workflow complet de gestion des bons cadeaux est opérationnel, de la création d'une demande jusqu'à la rédemption en magasin. La génération automatique des PDF avec QR codes uniques fonctionne correctement, de même que l'envoi automatique par email. Le module de rédemption avec détection de la double utilisation est fiable, et le module de reporting permet d'exporter les données aux formats souhaités.

7.2 Tests réalisés

J'ai effectué des tests fonctionnels sur chaque fonctionnalité au fur et à mesure du développement. Pour le module de rédemption, j'ai notamment testé les cas aux limites : scan d'un bon valide, scan d'un bon déjà utilisé, scan d'un bon expiré, et saisie d'un code inexistant. Tous ces cas sont gérés et produisent un retour visuel clair pour l'utilisateur.

J'ai également vérifié que le trigger PostgreSQL se déclenche bien en cas de tentative de double utilisation, et que le log d'audit est correctement créé dans la table correspondante.

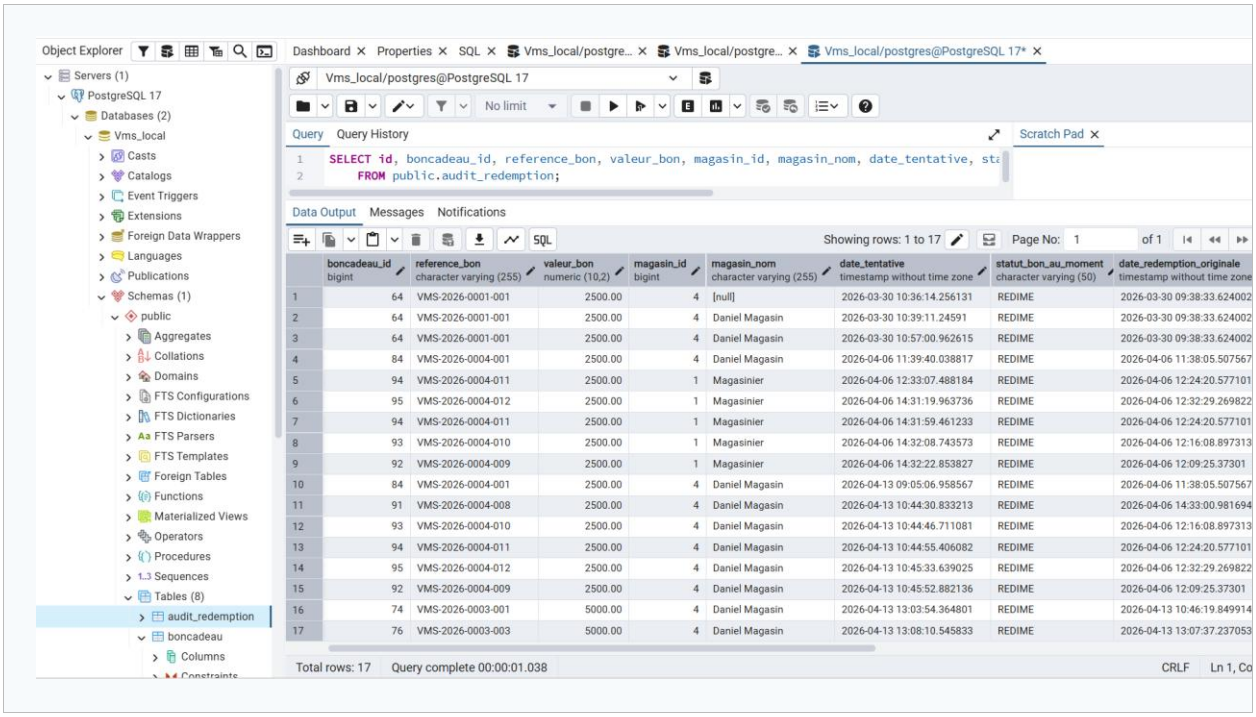

Figure 16 — Journal d'audit en base de données

7.3 Déploiement

L'application est distribuée sous forme d'un fichier exécutable Windows (VMS_Ingrid-1.0.0.exe) généré via un outil de packaging Java. L'installation ne nécessite pas de JDK sur le poste client puisque le runtime Java est inclus dans le package. La seule condition requise est une connexion Internet pour accéder à la base de données distante.

8. Bilan

8.1 Ce que j'ai appris

Ce projet a été une expérience formatrice sur plusieurs aspects. D'un point de vue technique, j'ai approfondi ma maîtrise de JavaFX et de l'architecture MVC, que j'avais abordés en cours mais sans jamais les avoir appliqués à un projet d'une telle envergure. J'ai également découvert et utilisé plusieurs bibliothèques professionnelles que je ne connaissais pas avant ce projet, comme PDFBox, ZXing et Apache POI.

La conception de la base de données a été une autre source d'apprentissage important. Réfléchir à la structure des tables, aux contraintes d'intégrité et à l'écriture de triggers m'a permis de mieux comprendre comment une base de données relationnelle est exploitée dans un contexte réel.

Sur le plan de l'organisation personnelle, conduire un projet seule sur six mois m'a appris à planifier mon travail, à tenir des délais et à prendre des décisions techniques de manière autonome.

8.2 Axes d'amélioration

Avec du recul, quelques points pourraient être améliorés. L'absence de tests unitaires formalisés est l'un des points faibles du projet : j'ai réalisé les tests manuellement, ce qui est moins fiable et reproductible que des tests automatisés avec JUnit. Par ailleurs, l'application repose sur une architecture de bureau alors qu'une interface web permettrait un accès plus souple depuis n'importe quel poste. Ce pourrait être une évolution envisageable à terme.

8.3 Compétences développées

Ce projet m'a permis de travailler les trois blocs de compétences du référentiel BTS SIO. J'ai conçu et développé une application complète de la conception jusqu'au déploiement, j'ai géré les données dans une base PostgreSQL avec des accès sécurisés, et j'ai assuré une forme de maintenance évolutive en ajoutant des fonctionnalités au cours du développement, comme le module de reporting, qui n'était pas prévu initialement dans le périmètre de base.

Annexes

Annexe A — Comptes de test

Les comptes suivants permettent de tester l'application avec les différents rôles disponibles :

Rôle	Identifiant	Mot de passe
SuperUser	Bilounga	123456789
Approbateur	TestAppro63d8c327	appro123
Initiateur	`Marie_Initiatrice_320b6cdf`	init123
Admin (magasin)	admin	123456789

Annexe B — Ressources du projet

Dépôt GitHub : https://github.com/raphafo/E6_VMS.git

Exécutable : VMS_Ingrid-1.0.0.exe (disponible dans le dépôt)

Contenu du dépôt : code source, diagrammes UML, rapport de projet, exécutable, Schéma sql

Annexe C — Diagrammes UML

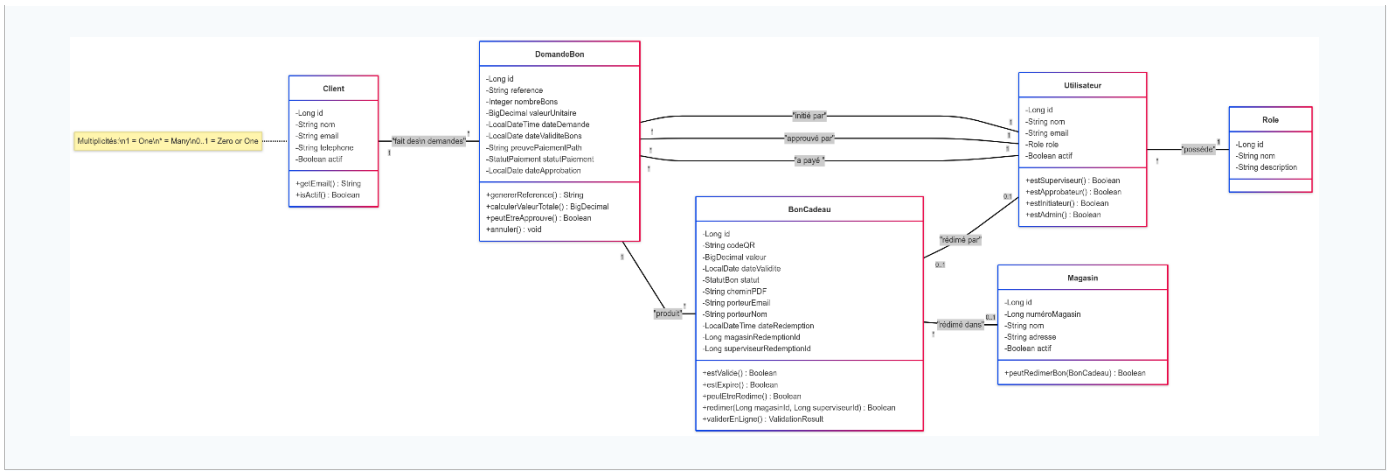

Figure A1 — Diagramme de classes

Fin du rapport